**UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA**
Facultad de Ingeniería de Producción y Servicios
Escuela Profesional de Ingeniería de Sistemas

## Sistemas Operativos

Semestre VI

**PRÁCTICA SEMANA 9**

# Almacenamiento y E/S

Copiador verificable y observable — Semana 9

**PRÁCTICA SEMANA 9**

Ruta de E/S, buffering, caché y escritura robusta de archivos

**AUTOR**
**Gustavo Alonso Carrillo Villalta**

**DOCENTE**
**Norman Patrick Harvey Arce**

**PERIODO ACADÉMICO**
**2026-B**

Arequipa – Perú
**2026**

# 1. Introducción

Esta práctica construye una utilidad de copia de archivos con tamaño de bloque configurable y la usa para recorrer la ruta completa de una operación de E/S: desde la llamada del proceso hasta el dispositivo, pasando por la page cache, la cola de bloque y el driver. El criterio central es que la utilidad preserve el destino válido ante cualquier fallo y que cada resultado parcial quede explicado.

La utilidad procesa lecturas y escrituras parciales, copia a un archivo temporal, reporta progreso sin saturar la salida, maneja falta de espacio y cancelación, verifica tamaño y hash, reemplaza el destino solo al completar y limpia los temporales ante error. Se implementó en Python 3.12 sobre las llamadas de bajo nivel `os.read`, `os.write` y `os.fsync`, que exponen el comportamiento real del sistema operativo (a diferencia de los archivos con buffer de alto nivel, que ocultan los resultados parciales).

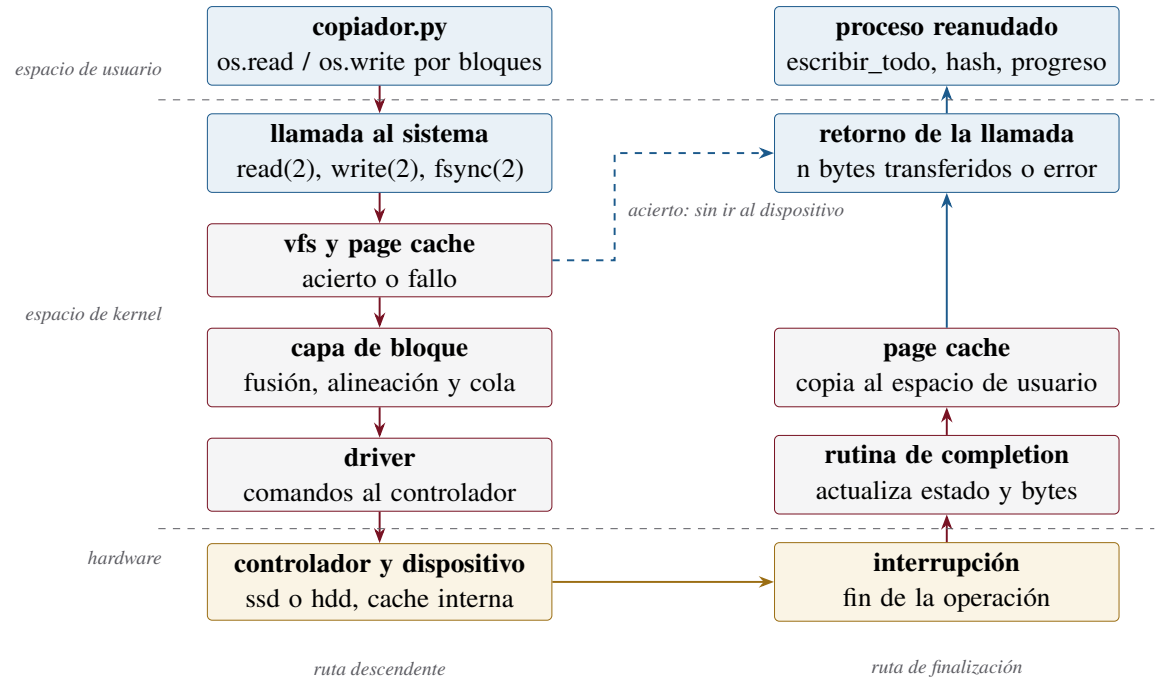
Todo el código fue ejecutado y las cifras provienen de esas ejecuciones; los datos crudos están en `resultados/medicion.csv` y `resultados/resumen.txt` del repositorio.

# 2. Diseño

## 2.1. Requisitos y mecanismos

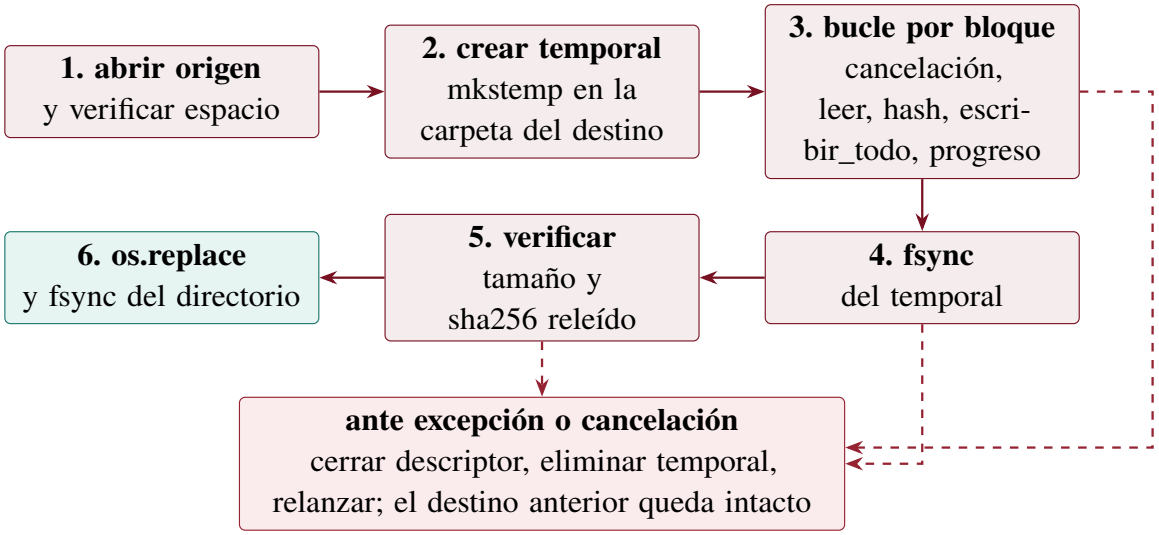
| Requisito | Mecanismo | Prueba que lo cubre |
|---|---|---|
| Lecturas y escrituras parciales | `escribir_todo()` avanza una `memoryview` con el valor devuelto por `os.write` hasta agotar el bloque; la lectura se repite hasta que `os.read` devuelve vacío | escrituras y lecturas parciales, escritura que devuelve 0 |
| Archivo temporal | `tempfile.mkstemp` en la misma carpeta del destino, para que el reemplazo sea un `rename` dentro del mismo sistema de archivos | destino reemplazado solo al completar |
| Progreso sin saturar | el callback se invoca como máximo cada `intervalo` segundos (`time.monotonic`) más un reporte final forzado | progreso no satura la salida |
| Falta de espacio | verificación previa con `shutil.disk_usage` y propagación de `ENOSPC` si el disco se llena durante la escritura | espacio detectado antes y durante |
| Cancelación | `threading.Event` consultado en cada bloque; la CLI lo activa con `SIGINT` y `SIGTERM` | cancelación cooperativa |
| Tamaño y hash | se compara el tamaño de origen, bytes copiados y temporal; el sha256 de lo leído se compara con el sha256 de una relectura del temporal | corrupción detectada por hash |
| Reemplazo atómico | `os.replace` y `fsync` del directorio | destino reemplazado solo al completar |
| Limpieza ante error | `except BaseException`: cierra el descriptor, elimina el temporal y relanza | todas las pruebas de fallo verifican que no queden `.tmp` |

## 2.2. Diagrama de la ruta de E/S




Cada cruce de frontera y cada cola es un punto donde se puede medir latencia. En la copia, una escritura con éxito solo garantiza que los datos llegaron a la page cache; la persistencia ocurre cuando se ejecuta `fsync` y el dispositivo confirma. Por eso el programa mide por separado la copia, el `fsync` y la verificación.

## 2.3. Flujo del algoritmo



## 2.4. Implementación

La función `copiar()` acepta las funciones de lectura y escritura como parámetros (`leer`, `escribir`). Por defecto son `os.read` y `os.write`; las pruebas las sustituyen para simular escrituras cortas, errores de E/S, disco lleno y corrupción sin tocar el sistema de archivos real.

Listing 1: copiador.py

```python
import argparse
import errno
import hashlib
import os
import shutil
import signal
import stat
import sys
import tempfile
import threading
import time
from dataclasses import dataclass


class cancelado(Exception):
    pass


class error_verificacion(Exception):
    pass


@dataclass
class resultado:
    bytes_copiados: int = 0
    sha256: str = ""
    lecturas: int = 0
    lecturas_cortas: int = 0
    escrituras: int = 0
    escrituras_parciales: int = 0
    t_copia: float = 0.0
    t_fsync: float = 0.0
    t_verificacion: float = 0.0

    @property
    def t_total(self):
        return self.t_copia + self.t_fsync + self.t_verificacion


def escribir_todo(fd, datos, escribir, res):
    vista = memoryview(datos)
    while vista:
        n = escribir(fd, vista)
        res.escrituras += 1
        if n is None or n <= 0:
            raise OSError(errno.EIO, "escritura incompleta")
        if n < len(vista):
            res.escrituras_parciales += 1
        vista = vista[n:]


def hash_fd(fd, bloque):
    h = hashlib.sha256()
```

```
    os.lseek(fd, 0, os.SEEK_SET)
    while True:
        datos = os.read(fd, bloque)
        if not datos:
            return h.hexdigest()
        h.update(datos)


def soltar_cache(fd):
    if hasattr(os, "posix_fadvise"):
        os.posix_fadvise(fd, 0, 0, os.POSIX_FADV_DONTNEED)


def verificar_espacio(carpeta, necesario):
    if shutil.disk_usage(carpeta).free < necesario:
        raise OSError(errno.ENOSPC, "espacio insuficiente en " + carpeta)


def copiar(origen, destino, bloque=65536, progreso=None, intervalo=0.5, cancelar=None,
           leer=os.read, escribir=os.write, usar_fsync=True, releer_disco=True):
    if bloque <= 0:
        raise ValueError("el bloque debe ser positivo")
    carpeta = os.path.dirname(os.path.abspath(destino))
    res = resultado()
    h = hashlib.sha256()
    fd_origen = os.open(origen, os.O_RDONLY)
    try:
        info = os.fstat(fd_origen)
        total = info.st_size
        verificar_espacio(carpeta, total)
        fd_tmp, ruta_tmp = tempfile.mkstemp(dir=carpeta, prefix="." + os.path.basename(destino) + ".",
      suffix=".tmp")
        try:
            os.fchmod(fd_tmp, stat.S_IMODE(info.st_mode))
            ultimo = 0.0

            def avisar(forzar=False):
                nonlocal ultimo
                if progreso is None:
                    return
                ahora = time.monotonic()
                if forzar or ahora - ultimo >= intervalo:
                    ultimo = ahora
                    progreso(res.bytes_copiados, total)

            ini = time.perf_counter()
            while True:
                if cancelar is not None and cancelar.is_set():
                    raise cancelado()
                datos = leer(fd_origen, bloque)
                if not datos:
                    break
                res.lecturas += 1
                if len(datos) < bloque and res.bytes_copiados + len(datos) < total:
                    res.lecturas_cortas += 1
                h.update(datos)
                escribir_todo(fd_tmp, datos, escribir, res)
                res.bytes_copiados += len(datos)
                avisar()
            res.t_copia = time.perf_counter() - ini
            avisar(True)

            ini = time.perf_counter()
```

```
            if usar_fsync:
                os.fsync(fd_tmp)
            res.t_fsync = time.perf_counter() - ini

            ini = time.perf_counter()
            if usar_fsync and releer_disco:
                soltar_cache(fd_tmp)
            res.sha256 = h.hexdigest()
            if os.fstat(fd_tmp).st_size != res.bytes_copiados or res.bytes_copiados != total:
                raise error_verificacion("tamano distinto: origen=%d copiado=%d temporal=%d"
                                         % (total, res.bytes_copiados, os.fstat(fd_tmp).st_size))
            if hash_fd(fd_tmp, bloque) != res.sha256:
                raise error_verificacion("hash distinto entre origen y temporal")
            res.t_verificacion = time.perf_counter() - ini

            os.close(fd_tmp)
            fd_tmp = None
            os.replace(ruta_tmp, destino)
            ruta_tmp = None
            if usar_fsync:
                fd_dir = os.open(carpeta, os.O_RDONLY)
                try:
                    os.fsync(fd_dir)
                finally:
                    os.close(fd_dir)
            return res
        except BaseException:
            if fd_tmp is not None:
                os.close(fd_tmp)
            if ruta_tmp is not None and os.path.exists(ruta_tmp):
                os.unlink(ruta_tmp)
            raise
    finally:
        os.close(fd_origen)


def parsear_bloque(texto):
    texto = texto.strip().lower()
    mult = {"k": 1024, "m": 1024 * 1024}
    if texto[-1] in mult:
        return int(texto[:-1]) * mult[texto[-1]]
    return int(texto)


def main(argv=None):
    ap = argparse.ArgumentParser(description="copia de archivos con verificacion y reemplazo atomico")
    ap.add_argument("origen")
    ap.add_argument("destino")
    ap.add_argument("--bloque", default="64k", help="tamano de bloque, ej: 4k, 64k, 1m")
    ap.add_argument("--intervalo", type=float, default=0.5, help="segundos entre reportes de progreso")
    ap.add_argument("--sin-progreso", action="store_true")
    ap.add_argument("--sin-fsync", action="store_true")
    a = ap.parse_args(argv)

    evento = threading.Event()
    signal.signal(signal.SIGINT, lambda *_: evento.set())
    signal.signal(signal.SIGTERM, lambda *_: evento.set())

    def mostrar(hecho, total):
        pct = 100.0 if total == 0 else 100.0 * hecho / total
        sys.stderr.write("\r%d/%d bytes (%.1f%%)" % (hecho, total, pct))
```

```
177         sys.stderr.flush()
178 
179     try:
180         r = copiar(a.origen, a.destino, parsear_bloque(a.bloque),
181                    None if a.sin_progreso else mostrar, a.intervalo, evento, usar_fsync=not a.
     sin_fsync)
182     except cancelado:
183         sys.stderr.write("\ncancelado: destino intacto y temporal eliminado\n")
184         return 130
185     except (OSError, error_verificacion) as e:
186         sys.stderr.write("\nerror: %s (destino intacto y temporal eliminado)\n" % e)
187         return 1
188     sys.stderr.write("\n")
189     print("bytes=%d sha256=%s" % (r.bytes_copiados, r.sha256))
190     print("lecturas=%d (cortas=%d) escrituras=%d (parciales=%d)"
191           % (r.lecturas, r.lecturas_cortas, r.escrituras, r.escrituras_parciales))
192     print("t_copia=%.4f t_fsync=%.4f t_verificacion=%.4f" % (r.t_copia, r.t_fsync, r.t_verificacion)
      )
193     return 0
194 
195 
196 if __name__ == "__main__":
197     sys.exit(main())
```

# 3. Pruebas automatizadas

`test_copiador.py` contiene 15 pruebas con `unittest` (el mínimo exigido es seis). Todas las pruebas de fallo comprueban dos cosas: que el destino previo siga con su contenido original y que no quede ningún archivo `.tmp`.

| Prueba | Qué verifica |
|---|---|
| `archivo_vacio` | copia de 0 bytes: destino vacío, sin temporales |
| `archivo_pequeno` | 100 bytes, menor que el bloque: mismo sha256 |
| `archivo_mayor_al_buffer` | 1 000 003 bytes con bloque de 4096: 245 lecturas, mismo sha256 |
| `escrituras_parciales_se_compl` | `write` que escribe como máximo 7 bytes: la copia sigue correcta y se cuentan las escrituras parciales |
| `lecturas_parciales_se_completan` | `read` que devuelve como máximo 5 bytes: la copia sigue correcta |
| `escritura_que_devuelve_cero_f` | progreso nulo en `write`: `OSError`, destino intacto, sin temporales |
| `error_durante_escritura_preserva_destino` | `OSError` de E/S en la tercera escritura: destino previo intacto |
| `falta_de_espacio_en_escritura` | `ENOSPC` durante la escritura: se propaga, destino intacto |
| `falta_de_espacio_detectada_antes_de_copiar` | espacio libre simulado de 10 bytes: falla antes de crear el temporal |
| `cancelacion_cooperativa` | el evento se activa a mitad de la copia: `cancelado`, destino intacto |
| `corrupcion_detectada_por_hash` | un byte alterado en cada bloque escrito: `error_verificacion`, destino intacto |
| `destino_se_reemplaza_solo_al_` | en cada escritura, el destino aún contiene el contenido previo |
| `progreso_no_satura_la_salida` | 391 bloques con intervalo de 60 s: a lo sumo 2 reportes, el último con el total |
| `origen_inexistente_no_toca_de` | `FileNotFoundError`, destino y carpeta intactos |
| `bloque_invalido` | bloque 0 produce `ValueError` |

Listing 2: Ejecución de las pruebas

```
$ python3 -m unittest
...............
----------------------------------------------------------------------
Ran 15 tests in 0.047s

OK
```

## 3.1. Evidencia con la línea de comandos

Listing 3: Copia normal, cancelación con Ctrl+C y error de destino (64 MiB)

```
$ python3 copiador.py grande.bin copia.bin --bloque 64k --intervalo 0.1
```

```
22609920/67108864 bytes (33.7%)
44498944/67108864 bytes (66.3%)
67108864/67108864 bytes (100.0%)
bytes=67108864 sha256=baf892493c7f456643eacac8d23cf16e25a48ba19f440a6bb6094d5d4e184f83
lecturas=1024 (cortas=0) escrituras=1024 (parciales=0)
t_copia=0.2774 t_fsync=0.0515 t_verificacion=0.2174

$ python3 copiador.py grande.bin salida.bin --bloque 4k (ctrl+c a los 0.15 s)
codigo de salida: 130
25710592/67108864 bytes (38.3%)
cancelado: destino intacto y temporal eliminado
contenido de salida.bin: viejo
temporales restantes: 0

$ python3 copiador.py grande.bin noexiste/x.bin

error: [Errno 2] No such file or directory: '/tmp/demo/noexiste' (destino intacto y temporal
    eliminado)
codigo de salida: 1
```

El sha256 de origen y copia coincide (`sha256sum`). En la cancelación, el código de salida es 130, el archivo de destino conserva su contenido anterior y no queda ningún temporal. El reporte de progreso se emite aproximadamente cada 0.1 s y sobrescribe la misma línea, por lo que no inunda la salida.

# 4. Medición con tres tamaños de bloque

## 4.1. Plataforma y parámetros

| Parámetro | Valor |
|---|---|
| Lenguaje / runtime | Python 3.12.3 |
| Sistema | Linux 6.18, máquina virtual con 1 núcleo lógico y 4 GB de RAM |
| Almacenamiento | `/dev/vda`, ext4, disco virtualizado |
| Archivo de prueba | 128 MiB de bytes aleatorios (`os.urandom`) |
| Tamaños de bloque | 4 KiB, 64 KiB y 1 MiB |
| Repeticiones | 5 por escenario; se reporta la mediana |
| Hash | sha256 (`hashlib`), 373 MiB/s medidos en memoria |

**ADVERTENCIA METODOLÓGICA**

Las cifras corresponden a una máquina virtual de un solo núcleo con disco virtualizado: el `fsync` confirma contra el disco virtual y el anfitrión puede mantener su propia caché. En dos campañas completas, el tiempo total por tamaño de bloque varió cerca de 15 % entre una y otra, de modo que las diferencias menores a ese margen no son concluyentes. Los tiempos sirven para comparar escenarios entre sí, no como valores absolutos.

## 4.2. Resultados

**Utilidad completa** (copia con hash, `fsync` y verificación por relectura; mediana de 5):

| Métrica | 4 KiB | 64 KiB | 1 MiB |
|---|---|---|---|
| Lecturas = escrituras | 32 768 | 2 048 | 128 |
| Copia (s) | 0.5214 | 0.4993 | 0.4345 |
| `fsync` (s) | 0.1607 | 0.1130 | 0.1159 |
| Verificación (s) | 0.4846 | 0.4494 | 0.4189 |
| **Total (s)** | **1.1630** | **1.0811** | **0.9792** |
| Rendimiento total (MiB/s) | 110 | 118 | 131 |

**Solo el bucle de lectura y escritura** (sin hash, sin `fsync`, sin verificación; mediana de 5):

| Métrica | 4 KiB | 64 KiB | 1 MiB |
|---|---|---|---|
| Tiempo del bucle (s) | 0.1136 | 0.0460 | 0.0500 |
| Rendimiento (MiB/s) | 1 127 | 2 784 | 2 561 |

**Efectos de `fsync` y de la caché** (bloque de 64 KiB; mediana de 5):

| Escenario | Variante A | Variante B |
|---|---|---|
| Total sin `fsync` / con `fsync` | 0.8055 s | 0.9746 s |
| Verificación con caché caliente / caché soltada | 0.3698 s | 0.4209 s |
| Lectura del origen con caché soltada / caliente | 0.4590 s | 0.4015 s |

# 5. Análisis de buffering y caché

**Buffering en el programa: el tamaño de bloque.** El bloque es el buffer del espacio de usuario y fija cuántas veces se cruza la frontera del kernel. Con 4 KiB se hacen 32 768 pares `read`/`write`; con 1 MiB, 128. En el bucle puro, 4 KiB es 2.5 veces más lento que 64 KiB (0.1136 s frente a 0.0460 s), lo que equivale a unos 2 $\mu$s por par de llamadas (0,0676 s/30 720). Entre 64 KiB y 1 MiB no hay diferencia apreciable: el costo por llamada ya está amortizado y a partir de ahí el límite es el movimiento de datos. En la utilidad completa el efecto del bloque se diluye (1.163 s frente a 0.979 s, cerca del margen de variación) porque el bucle puro es una fracción pequeña del total.

**El hash domina en esta plataforma.** El sha256 procesa unos 373 MiB/s, mientras que el bucle de copia mueve más de 2 500 MiB/s. Calcular el hash al copiar y volver a calcularlo al verificar cuesta, en conjunto, bastante más que copiar. La verificación no es gratuita ni es un costo de E/S: es CPU. Con un disco más lento (por ejemplo un HDD) el peso relativo cambiaría, por eso el rendimiento solo es interpretable junto con la plataforma y el patrón de acceso.

**Page cache y durabilidad.** Un `write` exitoso significa que los datos están en la page cache del kernel, no en el medio persistente. El bucle de copia logra más de 2 500 MiB/s precisamente porque nunca espera al dispositivo. Al ejecutar `fsync` se paga la persistencia: 0.113 s para 128 MiB en este disco virtual, entre 11 % y 14 % del total, y el total sube de 0.8055 s a 0.9746 s al activarlo. Si la energía se cortara antes del `fsync`, la copia "exitosa" podría perderse o quedar parcial; por eso el programa solo hace el `os.replace` después del `fsync` y verificación, y sincroniza además el directorio para que el renombrado sea duradero.

**Qué verifica realmente la relectura.** Tras el `fsync`, el programa suelta las páginas del temporal con `posix_fadvise(DONTNEED)` para que el hash lea desde el dispositivo y no desde la caché. Esa relectura cuesta 0.4209 s frente a 0.3698 s con la caché caliente (cerca de 14 % más). Si la verificación leyera de la page cache, solo comprobaría lo que el propio proceso acaba de escribir y no detectaría un fallo de la ruta hacia el medio. Aun así, el disco virtual y su anfitrión pueden mantener una caché adicional que el sistema invitado no controla.

**Caché en la lectura del origen.** Con la caché del origen soltada, la lectura tarda 0.4590 s frente a 0.4015 s con el archivo caliente (cerca de 14 % más). La diferencia es modesta porque la lectura es secuencial y el kernel hace lectura anticipada (*readahead*), que oculta buena parte de la latencia; con accesos aleatorios pequeños la diferencia sería mucho mayor. Esto también implica que las repeticiones consecutivas favorecen a quien lee con caché caliente, por lo que se reporta la mediana y se separan los escenarios.

**Resultados parciales.** En archivos locales `read` y `write` rara vez devuelven menos bytes de los pedidos, pero el contrato lo permite (sockets, tuberías, señales, disco casi lleno). El bucle que avanza una `memoryview` con el valor devuelto evita reconstruir los bytes restantes (no copia el bloque) y trata un retorno de 0 o `None` como error en lugar de

iterar sin fin. Las pruebas con `write` limitado a 7 bytes y `read` limitado a 5 bytes confirman que el resultado sigue siendo idéntico.

### 5.1. ¿Qué cambia si la escritura solo alcanza la caché del sistema operativo?

**Rendimiento:** la escritura se completa a velocidad de memoria y el proceso sigue sin esperar al dispositivo; el costo se difiere hasta el `fsync` o hasta que el kernel decide escribir en segundo plano. **Durabilidad:** mientras los datos estén solo en la caché, una pérdida abrupta de energía puede perderlos o dejarlos incompletos, aunque `write` haya devuelto éxito. La utilidad resuelve esto con tres pasos ordenados: escribir en un temporal, `fsync` y reemplazo atómico, de modo que el destino siempre contiene la versión anterior completa o la nueva completa, nunca una mezcla.

## 6. Conclusiones

1. Una copia robusta no puede suponer que `read` y `write` transfieren todo lo pedido: debe avanzar con el número real de bytes y tratar el progreso nulo como error.
2. El patrón temporal, verificación, `fsync` y `rename` deja el destino siempre en un estado válido. Las 15 pruebas, incluidas las de error de E/S, disco lleno, cancelación y corrupción, confirman que el destino previo se conserva y no quedan temporales.
3. El tamaño de bloque importa por el número de cruces al kernel: 4 KiB es 2.5 veces más lento que 64 KiB en el bucle puro, mientras que 64 KiB y 1 MiB son equivalentes en esta plataforma.
4. El rendimiento solo se interpreta junto con el patrón de acceso, la caché y el contrato de durabilidad: en esta máquina el sha256 y no el disco es el principal costo de la utilidad completa, y el `fsync` añade un costo medible a cambio de persistencia.
5. Una verificación honesta debe leer lo que realmente hay en el medio, no lo que la caché recuerda; por eso se suelta la caché del temporal antes de calcular el hash.

# Referencias bibliográficas

[1] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.ª ed. Boston, MA, USA: Pearson, 2015.

[3] Python Software Foundation, "os — Miscellaneous operating system interfaces", *Python 3 Documentation*.

Disponible en: https://docs.python.org/3/library/os.html

[4] Python Software Foundation, "unittest — Unit testing framework", *Python 3 Documentation*.

Disponible en: https://docs.python.org/3/library/unittest.html

[5] Michael Kerrisk, *Linux man-pages*, "read(2)", "write(2)", "fsync(2)", "rename(2)" y "posix_fadvise(2)", Linux man-pages project.